

RAMDEOBABA UNIVERSITY NAGPUR



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Student Name :- Advait Kawale

Section :- A4 (B3)

Roll No :- 42

Course Name :- Design and Analysis of Algorithms

Course Code :- 24CS01TH0201

Title :- Practical 5

Practical 5

Aim :- Implement a dynamic algorithm for Longest Common Subsequence (LCS) to find the length and LCS for DNA sequences.

Task 1 :- Find the similarity between the given X and Y sequence.

X=AGCCCTAAGGGCTACCTAGCTT

Y= GACAGCCTACAAGCGTTAGCTTG

Ans)

Input)

```
#include <stdio.h>
```

```
void printLCS(char b[100][100],char x[],int i,int j)
```

```
{
```

```
    if (i==0 || j==0)
```

```
    {
```

```
        return ;
```

```
    }
```

```
    if (b[i][j]=='d')
```

```
    {
```

```
        printLCS(b,x,i-1,j-1);
```

```
        printf("%c",x[i-1]);
```

```
    }
```

```
    else if (b[i][j]=='u')
```

```
    {
```

```
        printLCS(b,x,i-1,j);
```

```
    }
```

```
    else
```

```
    {
```

```
        printLCS(b,x,i,j-1);
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

```
    char
```

```
    x[]={'A','G','C','C','C','T','A','A','G','G','G','C','T','A','C','C','T','A','G','C','T','T'};
```

```
    char
```

```
    y[]={'G','A','C','A','G','C','C','T','A','C','A','A','G','C','G','T','T','A','G','C','T','T','G'}
```

```

;
int m=sizeof(x)/sizeof(x[0]);
int n=sizeof(y)/sizeof(y[0]);
int c[100][100];
char b[100][100];
for (int i=0;i<=m;i++)
{
    c[i][0]=0;
}
for (int i=0;i<=n;i++)
{
    c[0][i]=0;
}
for (int i=1;i<=m;i++)
{
    for (int j=1;j<=n;j++)
    {
        if (x[i-1]==y[j-1])
        {
            c[i][j]=c[i-1][j-1]+1;
            b[i][j]='d';
        }
        else if (c[i-1][j]>=c[i][j-1])
        {
            c[i][j]=c[i-1][j];
            b[i][j]='u';
        }
        else
        {
            c[i][j]=c[i][j-1];
            b[i][j]='l';
        }
    }
}
printf("Length of LCS = %d",c[m][n]);
printf("\nLCS = ");
printLCS(b,x,m,n);
}

```

```

1  #include <stdio.h>
2  void printLCS(char b[100][100],char x[],int i,int j)
3  {
4      if (i==0 || j==0)
5      {
6          return ;
7      }
8      if (b[i][j]=='d')
9      {
10         printLCS(b,x,i-1,j-1);
11         printf("%c",x[i-1]);
12     }
13     else if (b[i][j]=='u')
14     {
15         printLCS(b,x,i-1,j);
16     }
17     else
18     {
19         printLCS(b,x,i,j-1);
20     }
21 }
22 int main()
23 {
24     char x[]={'A','G','C','C','C','T','A','A','G','G','G','C','T','A','C','C','T','A','G','C','T','T'};
25     char y[]={'G','A','C','A','G','C','C','T','A','C','A','A','G','C','G','T','T','A','G','C','T','T','G'};
26     int m=sizeof(x)/sizeof(x[0]);
27     int n=sizeof(y)/sizeof(y[0]);
28     int c[100][100];
29     char b[100][100];
30     for (int i=0;i<=m;i++)
31     {
32         c[i][0]=0;
33     }
34     for (int i=0;i<=n;i++)
35     {
36         c[0][i]=0;
37     }
38     for (int i=1;i<=m;i++)
39     {
40         for (int j=1;j<=n;j++)
41         {
42             if (x[i-1]==y[j-1])
43             {
44                 c[i][j]=c[i-1][j-1]+1;
45                 b[i][j]='d';
46             }
47             else if (c[i-1][j]>=c[i][j-1])
48             {
49                 c[i][j]=c[i-1][j];
50                 b[i][j]='u';
51             }
52             else
53             {
54                 c[i][j]=c[i][j-1];
55                 b[i][j]='l';
56             }
57         }
58     }
59     printf("Length of LCS = %d",c[m][n]);
60     printf("\nLCS = ");
61     printLCS(b,x,m,n);
62 }

```

Output)

```

Length of LCS = 16
LCS = AGCCCAAGGTTAGCTT

```

Task 2 :- Find the longest repeating subsequence (LRS). Consider it as a variation of the longest common subsequence (LCS) problem.

Let the given string be S. You need to find the LRS within S. To use the LCS framework, you effectively compare S with itself. So, consider string1 = S and string2 = S.

Example:

AABCBDC

LRS = ABC or ABD

Ans)

Input)

```
#include <stdio.h>
```

```
void printLRS(char b[100][100],char x[],int i,int j)
```

```
{
    if(i==0||j==0)
    {
        return;
    }
    if(b[i][j]=='d')
    {
        printLRS(b,x,i-1,j-1);
        printf("%c",x[i-1]);
    }
    else if(b[i][j]=='u')
    {
        printLRS(b,x,i-1,j);
    }
    else
    {
        printLRS(b,x,i,j-1);
    }
}
```

```
int main()
```

```
{
    char x[]={'A','A','B','C','B','D','C'};
    int n=sizeof(x)/sizeof(x[0]);
    int c[100][100];
    char b[100][100];
    for(int i=0;i<=n;i++)
```

```

{
    c[i][0]=0;
}
for(int j=0;j<=n;j++)
{
    c[0][j]=0;
}
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=n;j++)
    {
        if(x[i-1]==x[j-1]&& i!=j)
        {
            c[i][j]=c[i-1][j-1]+1;
            b[i][j]='d';
        }
        else if(c[i-1][j]>=c[i][j-1])
        {
            c[i][j]=c[i-1][j];
            b[i][j]='u';
        }
        else
        {
            c[i][j]=c[i][j-1];
            b[i][j]='l';
        }
    }
}
printf("Length of LRS = %d",c[n][n]);
printf("\nLRS = ");
printLRS(b,x,n,n);
}

```

```

1  #include <stdio.h>
2  void printLRS(char b[100][100],char x[],int i,int j)
3  {
4      if(i==0||j==0)
5      {
6          return;
7      }
8      if(b[i][j]=='d')
9      {
10         printLRS(b,x,i-1,j-1);
11         printf("%c",x[i-1]);
12     }
13     else if(b[i][j]=='u')
14     {
15         printLRS(b,x,i-1,j);
16     }
17     else
18     {
19         printLRS(b,x,i,j-1);
20     }
21 }
22 int main()
23 {
24     char x[]={'A','A','B','C','B','D','C'};
25     int n=sizeof(x)/sizeof(x[0]);
26     int c[100][100];
27     char b[100][100];
28     for(int i=0;i<=n;i++)
29     {
30         c[i][0]=0;
31     }
32     for(int j=0;j<=n;j++)
33     {
34         c[0][j]=0;
35     }
36     for(int i=1;i<=n;i++)
37     {
38         for(int j=1;j<=n;j++)
39         {
40             if(x[i-1]==x[j-1]&&i!=j)
41             {
42                 c[i][j]=c[i-1][j-1]+1;
43                 b[i][j]='d';
44             }
45             else if(c[i-1][j]>=c[i][j-1])
46             {
47                 c[i][j]=c[i-1][j];
48                 b[i][j]='u';
49             }
50             else
51             {
52                 c[i][j]=c[i][j-1];
53                 b[i][j]='l';
54             }
55         }
56     }
57     printf("Length of LRS = %d",c[n][n]);
58     printf("\nLRS = ");
59     printLRS(b,x,n,n);
60 }

```

Output)

```

Length of LRS = 3
LRS = ABC

```

Task 3 :- LeetCode Assessment

1143. Longest Common Subsequence

Given two strings text1 and text2, return the length of their longest common subsequence. If there is no common subsequence, return 0.

A subsequence of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, "ace" is a subsequence of "abcde".

A common subsequence of two strings is a subsequence that is common to both strings.

Ans)

Input)

```
int max(int a,int b)
{
    if(a>b)
    {
        return a;
    }
    else
    {
        return b;
    }
}

int strlen_custom(char *s)
{
    int len=0;
    while(s[len]!='\0')
    {
        len++;
    }
    return len;
}

int longestCommonSubsequence(char* text1,char* text2)
{
    int m=strlen_custom(text1);
    int n=strlen_custom(text2);
```



```
int dp[m+1][n+1];

for(int i=0;i<=m;i++)
{
    for(int j=0;j<=n;j++)
    {
        if(i==0 || j==0)
        {
            dp[i][j]=0;
        }
        else if(text1[i-1]==text2[j-1])
        {
            dp[i][j]=dp[i-1][j-1]+1;
        }
        else
        {
            dp[i][j]=max(dp[i-1][j],dp[i][j-1]);
        }
    }
}
return dp[m][n];
}
```

```

1  int max(int a,int b)
2  {
3      if(a>b)
4      {
5          return a;
6      }
7      else
8      {
9          return b;
10     }
11 }
12 int strlen_custom(char *s)
13 {
14     int len=0;
15     while(s[len]!='\0')
16     {
17         len++;
18     }
19     return len;
20 }
21 int longestCommonSubsequence(char* text1,char* text2)
22 {
23     int m=strlen_custom(text1);
24     int n=strlen_custom(text2);
25     int dp[m+1][n+1];
26
27     for(int i=0;i<=m;i++)
28     {
29         for(int j=0;j<=n;j++)
30         {
31             if(i==0 || j==0)
32             {
33                 dp[i][j]=0;
34             }
35             else if(text1[i-1]==text2[j-1])
36             {
37                 dp[i][j]=dp[i-1][j-1]+1;
38             }
39             else
40             {
41                 dp[i][j]=max(dp[i-1][j],dp[i][j-1]);
42             }
43         }
44     }
45     return dp[m][n];
46 }

```

Output)

The screenshot shows a coding platform interface with two main panels. The left panel displays submission details for a problem titled "Longest Common Subsequence". The submission is marked as "Accepted" with 47/47 testcases passed. The runtime is 21 ms, which is 77.30% better than the rest, and the memory usage is 12.48 MB, which is 30.17% better. A bar chart shows the distribution of runtimes for other submissions. The right panel shows the code for the submission, which is a C++ implementation of the Longest Common Subsequence algorithm using dynamic programming. The code defines a `max` function, a custom `strlen` function, and the main `longestCommonSubsequence` function. The test results for Case 1 are shown below the code, with input strings "abcde" and "ace", and the expected output is 3.

Submission Details:

- Status: Accepted
- Testcases: 47 / 47 testcases passed
- Runtime: 21 ms (Beat: 77.30%)
- Memory: 12.48 MB (Beat: 30.17%)

Code:

```

1  int max(int a,int b)
2  {
3      if(a>b)
4      {
5          return a;
6      }
7      else
8      {
9          return b;
10     }
11 }
12 int strlen_custom(char *s)
13 {
14     int len=0;
15     while(s[len]!='\0')
16     {
17         len++;
18     }
19     return len;
20 }
21 int longestCommonSubsequence(char* text1,char* text2)
22 {
23     int m=strlen_custom(text1);
24     int n=strlen_custom(text2);
25     int dp[m+1][n+1];
26
27     for(int i=0;i<=m;i++)
28     {
29         for(int j=0;j<=n;j++)
30         {
31             if(i==0 || j==0)
32             {
33                 dp[i][j]=0;
34             }
35             else if(text1[i-1]==text2[j-1])
36             {
37                 dp[i][j]=dp[i-1][j-1]+1;
38             }
39             else
40             {
41                 dp[i][j]=max(dp[i-1][j],dp[i][j-1]);
42             }
43         }
44     }
45     return dp[m][n];
46 }

```

Test Results:

Accepted Runtime: 0 ms

Case 1

Input:

text1 = "abcde"

text2 = "ace"

Output:

3

Expected:

3